Live Site Reliability Project

Observability and resilience for always-on services, explained in plain language

> **Project statement:** Reduced Live Site MTTR 40% at 99.9% availability by instrumenting telemetry, structured logging, alerting, retries, and circuit breakers for actively monitored, diagnosable services.

1. The Big Picture (Simple Summary)

Imagine a busy online shop that must stay open 24 hours a day. Sometimes things break: a database slows down, a partner's website stops answering, or a new code change has a bug. When that happens, customers see errors until engineers find the problem and fix it.

The time from “something broke” to “it is working again” is called **MTTR (Mean Time To Recover)**. This project cut that time by **40%** while keeping the service up **99.9%** of the time. It did this in two ways:

- **Make problems easy to see and understand (observability).** Every service now reports health numbers (telemetry), writes clean, searchable notes about what it is doing (structured logging), and automatically calls an engineer when something looks wrong (alerting). Engineers stop guessing and go straight to the cause.
- **Make services handle small problems on their own (resilience).** If a call fails for a moment, the service tries again carefully (retries). If a dependency is truly down, the service stops hammering it and fails fast with a fallback (circuit breaker). Many small issues never become outages, and big ones stay contained.

Put simply: problems are **found faster**, **understood faster**, and **contained faster**, so customers are affected for less time.

1.1 Key terms in one line each

Term	Simple meaning
Live Site	The real, production system that customers are using right now (not a test copy).

Term	Simple meaning
MTTR	Mean Time To Recover: the average time from when a problem starts to when service is back to normal.
Availability 99.9%	The service works 99.9% of the time. That allows only about 43.8 minutes of downtime per month, or about 8.8 hours per year.
Telemetry	Numbers and signals a service sends out about its own health: how many requests, how many errors, how slow.
Structured logging	Writing log messages as organized fields (like a form) instead of free text, so computers can search and filter them.
Alerting	Automatic rules that watch the numbers and notify a person when something crosses a danger line.
Retry	Trying a failed operation again, with waiting time in between, because many failures are brief.
Circuit breaker	A safety switch that stops calling a broken dependency for a while, so the failure does not spread.
Actively monitored	Someone (or something) is always watching the service and will be told immediately if it misbehaves.
Diagnosable	When it breaks, the data needed to find out why is already there.

1.2 Before and after

Situation	Before the project	After the project
How problems were found	Often by customers complaining or by chance.	Automatic alerts fire within minutes based on error rate and latency.
How engineers investigated	Logged in to machines and read messy text logs one by one.	Opened a dashboard, followed one request ID across all services, searched logs by field.
What happened on a brief network blip	Request failed and the customer saw an error.	Request quietly retried and succeeded.
What happened when a dependency went down	Threads piled up waiting, the whole service slowed, and the failure spread.	Circuit breaker opened, calls failed fast with a fallback, the rest of the service stayed healthy.
After an incident	Same issue could repeat.	Postmortem added new alerts, runbook steps, and fixes.

1.3 What MTTR is made of

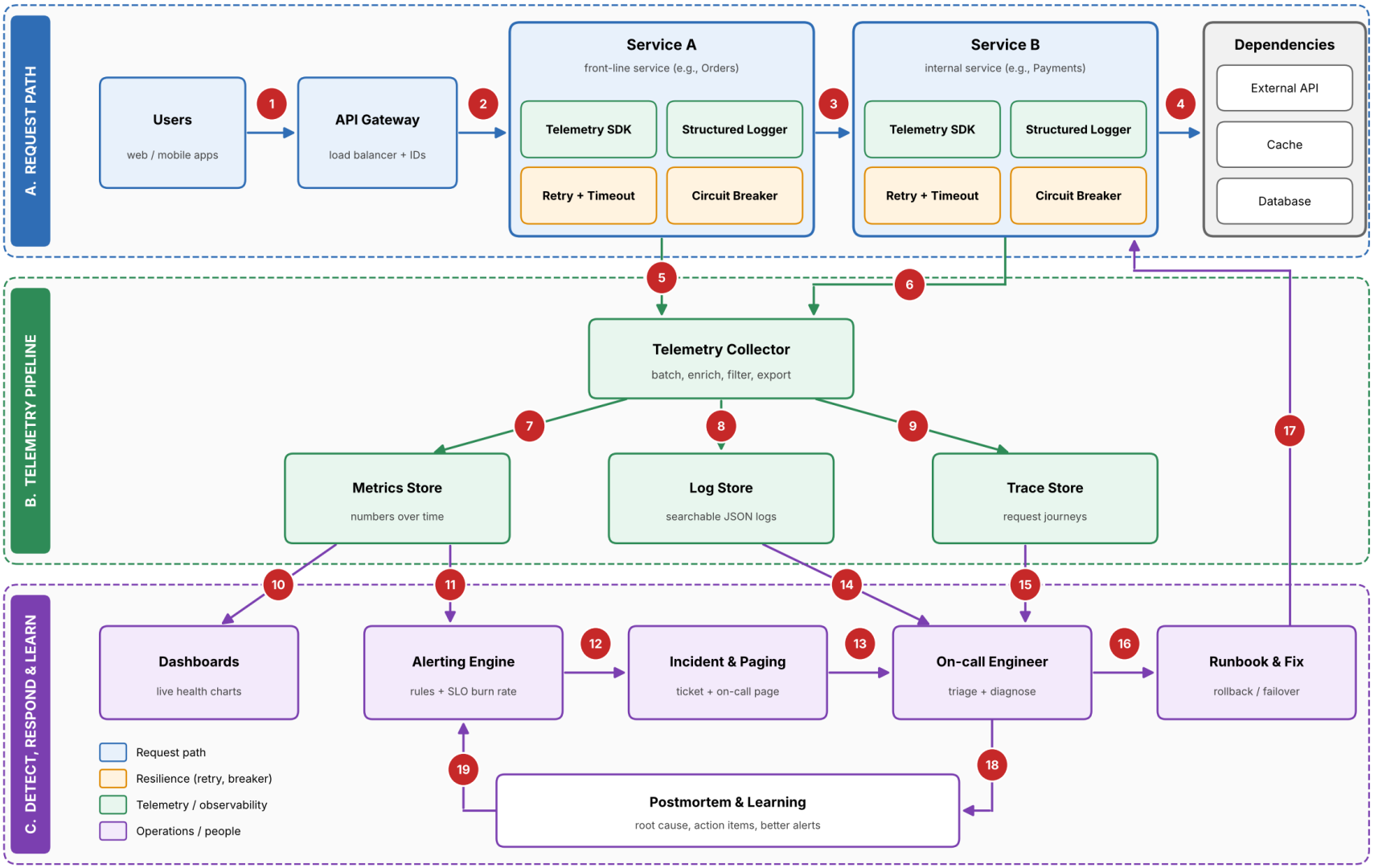
MTTR is not one block of time. It is the sum of smaller stages. The project attacked each stage:

Stage	Plain meaning	What reduced it
1. Detect	Noticing that something is wrong.	Telemetry + alerting (SLO burn-rate alerts).
2. Acknowledge	A person accepts the alert and starts working.	Paging with clear, actionable alerts and on-call rotation.
3. Diagnose	Finding out what is broken and why.	Structured logs, correlation IDs, traces, dashboards.
4. Mitigate	Stopping customer pain (roll back, fail over, switch off a feature).	Runbooks linked from alerts; circuit breakers limiting blast radius.
5. Recover / verify	Confirming everything is normal again.	Dashboards and the same metrics that raised the alert now show recovery.

> **Illustrative example:** If the average incident used to take 50 minutes (10 to detect, 5 to acknowledge, 25 to diagnose, 10 to fix) and now takes 30 minutes (3 to detect, 3 to acknowledge, 14 to diagnose, 10 to fix), that is a 40% reduction. These stage numbers are an example to show how the math works, not measured data.

2. Architecture Diagram

Read the diagram top to bottom. Layer A is where customer requests travel. Layer B collects and stores health signals. Layer C is how alerts reach people and how problems get fixed. Red numbered circles mark each arrow; Section 3 explains every box and every number.



3. The Diagram Explained, Piece by Piece

3.1 How to read it

- **Three horizontal layers (dashed boxes with a colored side label).** A = Request Path (blue), B = Telemetry Pipeline (green), C = Detect, Respond & Learn (purple).
- **Box colors** tell you the job of each piece: blue = handles customer requests, orange = resilience (retry, circuit breaker), green = telemetry and observability, purple = operations and people, grey = outside dependencies.
- **Arrows** show the direction something moves: a request, a signal, an alert, or a fix.
- **Red numbered circles** label each arrow so it can be explained below. Numbers roughly follow time order.
- **The legend** in the bottom-left corner repeats the color meanings.

3.2 Every box in the diagram

Layer	Box	What it does in simple words
A	**Users**	People using the website or mobile app. Every click becomes a request.
A	**API Gateway**	The front door. Spreads traffic across servers (load balancer), checks basic rules, and stamps every request with a unique ID.
A	**Service A**	The first business service the request reaches (example: Orders). Contains four built-in helpers.
A	**Service B**	A second service that Service A calls (example: Payments). Has the same four helpers.
A	**Telemetry SDK** (inside each service)	A small library that counts requests, errors and timing, and records the request's journey (traces).
A	**Structured Logger** (inside each service)	Writes every important event as a neat record with named fields such as time, level, request ID and error code.
A	**Retry + Timeout** (inside each service)	Waits only a set time for an answer, and if a call fails for a passing reason, tries again with growing pauses.
A	**Circuit Breaker** (inside each service)	Watches calls to a dependency. If too many fail, it stops calling it for a while and returns a fallback answer fast.

Layer	Box	What it does in simple words
A	**Dependencies**	Things the services rely on but do not control directly: the Database, the Cache, and External APIs (partners).
B	**Telemetry Collector**	A central mailroom for signals. Receives data from all services, groups it, adds labels, removes noise and private data, and sends it to the right store.
B	**Metrics Store**	A database of numbers over time (for example, errors per minute). Fast for charts and alert rules.
B	**Log Store**	A searchable library of all structured log records.
B	**Trace Store**	Keeps the full journey of individual requests across services, with timing for each step.
C	**Dashboards**	Live charts that show service health at a glance.
C	**Alerting Engine**	Constantly checks metrics against rules and reliability targets (SLOs). Fires an alert when something is wrong.
C	**Incident & Paging**	Opens an incident ticket and phones or pushes a notification to the on-call engineer.
C	**On-call Engineer**	The person on duty. Looks at the alert, dashboards, logs and traces to find the cause.
C	**Runbook & Fix**	A step-by-step guide for known problems and the action taken: roll back a release, fail over, change a setting, or scale up.
C	**Postmortem & Learning**	A blame-free review after the incident that produces improvements, such as better alerts and fixes.

3.3 Every numbered arrow

#	From → To	What travels along it, in simple words
1	Users → API Gateway	A customer's request (for example, “place my order”) arrives at the front door.
2	API Gateway → Service A	The gateway forwards the request to a healthy copy of Service A, adding a unique correlation ID (trace ID) to its headers.
3	Service A → Service B	Service A needs help (for example, take payment) and calls Service B. The call goes through A's timeout, retry and circuit-breaker helpers, and the same ID is passed along.

#	From → To	What travels along it, in simple words
4	Service B → Dependencies	Service B reads or writes the database, checks the cache, or calls an external partner, again guarded by its own timeout, retry and circuit breaker.
5	Service A → Telemetry Collector	Service A sends its metrics, logs and trace pieces in the background, without slowing the customer's request.
6	Service B → Telemetry Collector	Service B does the same. Because both used the same ID, their data can be joined later.
7	Collector → Metrics Store	Numbers such as request count, error count, latency, retry count and circuit-breaker state are saved.
8	Collector → Log Store	Structured log records are saved and indexed so they can be searched by any field.
9	Collector → Trace Store	Request journeys (spans with start and end times) are saved.
10	Metrics Store → Dashboards	Dashboards read the numbers and draw live charts.
11	Metrics Store → Alerting Engine	The alerting engine reads the same numbers every minute and compares them to rules and SLOs.
12	Alerting Engine → Incident & Paging	A rule is broken, so an alert is sent with a summary, severity, links to dashboards and the runbook.
13	Incident & Paging → On-call Engineer	The engineer is paged by phone or app and acknowledges the incident.
14	Log Store → On-call Engineer	The engineer searches logs by request ID, error code or service to see exactly what failed.
15	Trace Store → On-call Engineer	The engineer opens a trace to see which hop in the journey was slow or failed.
16	On-call Engineer → Runbook & Fix	With the cause known, the engineer follows the runbook to mitigate.
17	Runbook & Fix → Services	The fix is applied to the live services (rollback, failover, feature flag off, config change, scale out). The dashboards then show recovery.
18	On-call Engineer → Postmortem	After service is restored, the engineer writes up what happened, the timeline and the root cause.
19	Postmortem → Alerting Engine	Lessons go back into the system: new or tuned alerts, new runbook steps, new log fields. Next time is faster.

4. Each Component in Detail

Every component below is explained the same way: what it is, an everyday comparison, how it works step by step, the small details that matter, and why it helps this project.

4.1 Users and Client Apps

What it is: The web browsers and mobile apps customers use. They are the source of every request and the people whose experience defines “up” or “down”.

> **In everyday terms:** Customers walking into a shop. If the shop door is jammed, it does not matter how good the shelves are.

How it works, step by step

1. The user taps a button, for example “Buy now”.
2. The app builds an HTTPS request and sends it over the internet to the company's public address.
3. The app waits for a reply. If it gets an error, it may show a friendly message and offer to try again.
4. Optionally, the app itself reports client-side telemetry (page load time, app errors) so the team sees what users actually feel.

Small details that matter

- Availability is measured from the user's point of view: a request counts as “good” if it succeeds and is fast enough.
- Client apps should not retry aggressively on their own, or they can add to an overload. Retries are best controlled on the server side.

Why it helps this project

Defining success from the user's side keeps the 99.9% target honest. Every other component exists to protect this experience.

4.2 API Gateway and Load Balancer

What it is: The single front door for all traffic. It sends each request to a healthy server, applies basic rules, and labels every request with a unique ID.

> **In everyday terms:** A receptionist who greets every visitor, gives them a numbered ticket, and sends them to an available desk.

How it works, step by step

1. Receives the incoming request and checks it is allowed (authentication, size limits, rate limits).

2. Generates a **correlation ID** (also called a trace ID), a long unique string such as 4f2a9c…, and attaches it to the request headers. If the request already has one, it keeps it.
3. Checks which copies (instances) of Service A are healthy using regular health checks.
4. Forwards the request to one healthy instance, spreading load evenly.
5. Records its own metrics: requests per second, response codes (2xx, 4xx, 5xx), and time taken.
6. Returns the final response to the user and includes the correlation ID in the response so support staff can look it up if a customer reports a problem.

Small details that matter

- **Health checks** remove sick instances from rotation automatically, which is itself a recovery mechanism.
- **Rate limiting** protects services from sudden floods of traffic.
- The correlation ID is the thread that ties all later logs and traces together. Without it, diagnosis means guessing.

Why it helps this project

The correlation ID created here is what lets an engineer follow one failing request across every service in seconds, directly shortening the diagnose stage of MTTR.

4.3 Services (Service A and Service B)

What it is: The programs that do the real business work, such as creating orders or taking payments. Each runs as several copies for safety, and each contains four built-in helpers: telemetry SDK, structured logger, retry plus timeout, and circuit breaker.

> **In everyday terms:** Departments in a company. Orders talks to Payments, Payments talks to the bank. Each department keeps a diary (logs), reports numbers to management (telemetry), and has rules for what to do when another department does not answer (retry and circuit breaker).

How it works, step by step

1. Service A receives the request and reads the correlation ID from the header.
2. It starts a trace span (a timed record of its own work) and writes a log line such as “order request received”.
3. It does its own logic (validate the cart, calculate the total).
4. It calls Service B through a protected client that applies timeout, retry and circuit-breaker rules, passing the correlation ID along.
5. Service B repeats the same pattern and calls its dependencies.

6. Each service ends its span, records success or failure in metrics, writes a final log line, and returns the result.

Small details that matter

- Services run as **multiple instances** behind the load balancer, so one crashing copy does not take the service down.
- Every service exposes a **health endpoint** (for example /health) that reports whether it and its key dependencies are ready.
- The helpers are added as shared libraries or middleware, so every team gets the same behavior without writing it again.

Why it helps this project

Putting the same instrumentation and protection into every service is what makes them “actively monitored and diagnosable”. No service is a black box.

4.4 Telemetry SDK (Metrics and Traces)

What it is: A software library inside each service that automatically measures what the service is doing and sends those measurements out. It produces two main kinds of signal: **metrics** (numbers over time) and **traces** (the journey of one request).

> **In everyday terms:** A car's dashboard and its trip recorder. The dashboard shows speed and engine temperature (metrics). The trip recorder logs every stop on a journey with times (traces).

How it works, step by step

1. When the service starts, the SDK is configured with the service name, version, environment (for example, production) and region.
2. For every incoming request, it starts a **span**: a record with a start time, the correlation ID and the operation name.
3. For every outgoing call (to Service B or the database), it starts a **child span** linked to the parent, and passes the trace context to the next service in the headers.
4. When each call finishes, it records the end time, status (success or error) and useful attributes (HTTP status, retry count).
5. At the same time, it updates counters and histograms: total requests, failed requests, and how long requests took.
6. It batches all this data in memory and sends it to the Telemetry Collector every few seconds in the background.

Small details that matter

- **The four golden signals:** latency (how slow), traffic (how many), errors (how many failed), saturation (how full the CPU, memory or queues are).
- **RED method** for each endpoint: Rate, Errors, Duration.
- **Percentiles, not just averages:** p50 shows the typical user; p95 and p99 show the slowest users, who often reveal the problem first.
- **Custom resilience metrics:** number of retries, number of calls rejected by an open circuit breaker, and the breaker's current state. These show trouble before users feel it.
- **Low overhead:** sending is asynchronous and batched, and traces can be sampled (for example, keep 10% of normal traces but 100% of failed ones).
- **Labels (dimensions)** such as service, endpoint, region, version let you slice the data, for example "errors only in region West after version 2.3".

Signal	Example	Question it answers
Counter	http_requests_total = 1,204,330	How much traffic are we serving?
Counter	http_errors_total{code=500} = 412	How many requests failed?
Histogram	request_duration_ms p99 = 870 ms	How slow are the slowest users?
Gauge	circuit_breaker_state{dep=payments} = OPEN	Is a dependency currently cut off?
Trace span	Orders → Payments → DB, 1.2 s total, DB 1.0 s	Where in the journey is the time going?

Why it helps this project

Telemetry turns "the site feels slow" into "p99 latency on the checkout endpoint in region West jumped from 200 ms to 2 s at 10:42 after version 2.3". That precision is what shortens detection and diagnosis.

4.5 Structured Logger

What it is: A logging library that writes every log message as a set of named fields (usually JSON) instead of a free-text sentence.

> **In everyday terms:** Filling in a form instead of writing a letter. A form has the same boxes every time, so thousands of forms can be sorted and searched in seconds.

How it works, step by step

1. When something important happens (request received, call failed, retry attempted, breaker opened), the code calls the logger with a message and fields.
2. The logger automatically adds standard fields: timestamp, level, service name, version, host, and the correlation ID from the current request.
3. It removes or masks sensitive data such as passwords, card numbers and personal details.
4. It writes the record as one line of JSON to standard output or a local agent.
5. The record is shipped to the Telemetry Collector and then the Log Store, where every field becomes searchable.

Small details that matter

- **Unstructured example:** “Payment failed for user 123 after 3 tries”. Hard to search reliably.
- **Structured example:** {"ts":"2026-03-14T10:42:07Z", "level":"ERROR", "service":"payments", "trace_id":"4f2a9c", "event":"payment_failed", "dependency":"bank-api", "attempts":3, "error_code":"TIMEOUT", "duration_ms":3012}
- **Log levels:** DEBUG (detail for developers, usually off in production), INFO (normal events), WARN (something odd but handled, such as a retry), ERROR (a failure), FATAL (the service cannot continue).
- **Consistent field names** across all services (always trace_id, never traceId in one place and requestId in another), so one search works everywhere.
- **Error codes** that are stable and specific (for example, DB_POOL_EXHAUSTED) instead of only free-text messages.
- **Volume control:** sample repetitive success logs, but always keep warnings and errors.

Why it helps this project

An engineer can search “trace_id = 4f2a9c” and instantly see every log line from every service for that one failing request, in order. Diagnosis that used to take a long time of reading raw text now takes minutes.

4.6 Retry with Timeout

What it is: Two rules that sit around every call to another service or dependency. **Timeout:** only wait a fixed amount of time for an answer. **Retry:** if the call fails for a temporary reason, try again after a short, growing pause.

> **In everyday terms:** Calling a friend. If they do not pick up after 6 rings, you hang up (timeout). You try again in 1 minute, then 2 minutes, then 4 (backoff). After 3 tries you stop and text instead (give up gracefully).

How it works, step by step

1. The service makes a call with a **timeout**, for example 800 ms. If no answer arrives in time, the call is treated as failed instead of waiting forever.
2. If the call fails, the code checks whether the failure is **transient** (temporary): timeouts, connection resets, HTTP 503 or 429. Permanent errors such as 400 Bad Request or 404 Not Found are not retried.
3. It checks the operation is **idempotent**, meaning safe to repeat. Reading data is safe. Charging a card is only safe if an idempotency key is sent so the bank ignores duplicates.
4. It waits using **exponential backoff with jitter**: about 100 ms, then 200 ms, then 400 ms, each with a small random amount added so thousands of clients do not retry at the same instant.
5. It retries up to a **maximum number of attempts**, for example 3, and within an **overall deadline** for the whole request.
6. Every retry is logged as WARN and counted in a retry metric. If all attempts fail, the error is passed upward or to the circuit breaker.

Small details that matter

- **Retry storms:** if every layer retries 3 times, three layers can multiply one request into 27. The fix is to retry at one layer only, or use a **retry budget** (for example, retries may add at most 10% extra traffic).
- **Respect Retry-After headers** when a server says how long to wait.
- **Timeouts must shrink going deeper:** the outer request deadline is longer than inner call timeouts, so inner calls give up before the user does.

Setting	Typical example value	Plain meaning
Timeout per attempt	500 to 1,000 ms	How long to wait for one answer.
Max attempts	3	Total tries including the first.
Base backoff	100 ms	First wait before retrying.
Backoff multiplier	2x	Each wait doubles.
Jitter	± up to 50% of the wait	Random spread so clients do not retry together.
Retry on	Timeout, 503, 429, connection reset	Only temporary errors.

Why it helps this project

Most cloud failures are brief: a network blip, a server restarting, a short overload. Careful retries hide these from customers so they never become incidents, protecting the 99.9% availability target. Timeouts stop slow dependencies from freezing the whole service.

4.7 Circuit Breaker

What it is: A safety switch around calls to a dependency. When that dependency keeps failing, the breaker "trips" and stops sending calls to it for a while, returning a fast failure or fallback answer instead.

> **In everyday terms:** The electrical breaker in your house. If a faulty appliance keeps drawing too much power, the breaker cuts that circuit so the whole house does not catch fire. After fixing it, you flip it back on.

How it works, step by step

1. **CLOSED (normal):** calls flow through. The breaker counts successes and failures over a rolling window, for example the last 20 calls or 30 seconds.
2. **Trip condition:** if failures cross a threshold, for example 50% failure rate over at least 20 calls, or slow calls above a limit, the breaker switches to OPEN.
3. **OPEN (protecting):** calls are not sent at all. The service immediately returns a fallback: cached data, a default value, a "try again later" message, or queues the work for later. This takes microseconds instead of waiting for timeouts.
4. **Cool-down:** after a wait, for example 30 seconds, the breaker moves to HALF-OPEN.
5. **HALF-OPEN (testing):** a few trial calls are allowed through. If they succeed, the breaker goes back to CLOSED. If they fail, it returns to OPEN and waits again.
6. Every state change is logged and published as a metric, so dashboards and alerts know exactly which dependency is cut off.

Small details that matter

- **Why it matters:** without a breaker, every request waits for the full timeout on a dead dependency. Threads and connections pile up, the calling service runs out of resources, and the failure spreads (a cascading failure).
- **Fallbacks** must be designed per feature, for example show cached product prices, or accept the order and process payment later.
- **One breaker per dependency** (and sometimes per endpoint), so a broken partner API does not block database calls.

- Often paired with a **bulkhead**: a separate, limited pool of threads or connections per dependency, so one cannot use all resources.
- Retry and breaker work together: retries handle short blips; if retries keep failing, the breaker trips and stops the retrying.

State	Are calls sent?	What the user sees
CLOSED	Yes, all of them	Normal behavior.
OPEN	No, blocked instantly	A fast fallback or a clear, quick error instead of a long hang.
HALF-OPEN	Only a few test calls	Mostly fallback; normal service resumes once tests pass.

Why it helps this project

The breaker limits the **blast radius**: one broken dependency degrades one feature instead of taking the whole site down. Its state metric also tells the on-call engineer the exact dependency at fault, which shortens diagnosis.

4.8 Dependencies (Database, Cache, External APIs)

What it is: Systems the services rely on but that run separately: the database (permanent storage), the cache (fast temporary storage), and external APIs (partners such as a bank or shipping provider).

> **In everyday terms:** Suppliers to a shop. If the delivery truck is late, the shop must decide whether to wait, use stock on the shelf, or tell the customer to come back later.

How it works, step by step

1. The service calls the dependency through its protected client (timeout, retry, circuit breaker).
2. The telemetry SDK records a span and metrics for each call: duration, success or error, which dependency.
3. The cache is checked first for frequently read data; on a cache miss, the database is used.
4. If a dependency is slow or failing, the breaker and fallback logic decide what to return.

Small details that matter

- Database health signals worth watching: connection pool usage, query latency, deadlocks, replication lag.
- Cache signals: hit rate, eviction rate, latency.

- External API signals: error rate and latency per partner, rate-limit responses (429).

Why it helps this project

Most outages begin in a dependency. Measuring every dependency call separately means the on-call engineer sees “the bank API is timing out” directly instead of a vague “Payments is broken”.

4.9 Telemetry Collector

What it is: A central service (or agent) that receives metrics, logs and traces from all services, cleans and organizes them, and forwards each type to the correct storage system.

> **In everyday terms:** A post office sorting center. Letters arrive from everywhere, get stamped, sorted by type and destination, junk mail is removed, and each bag is sent to the right city.

How it works, step by step

1. **Receive:** accepts data from all services over standard protocols (for example, OpenTelemetry OTLP).
2. **Batch:** groups many small items together so sending is efficient.
3. **Enrich:** adds labels the service might not know, such as cluster, region, or Kubernetes pod name.
4. **Filter and sample:** drops noisy debug data, keeps all errors, samples successful traces.
5. **Scrub:** removes any personal or secret data that slipped through.
6. **Route and export:** sends metrics to the Metrics Store, logs to the Log Store, traces to the Trace Store.
7. **Buffer and retry:** if a store is briefly unavailable, it holds data in a queue and retries, so signals are not lost during the very incidents you need them for.

Small details that matter

- Decouples services from storage choices: you can change the log vendor without touching service code.
- Runs in more than one copy, and the collector itself is monitored (dropped data, queue size).

Why it helps this project

One consistent pipeline means all teams' data looks the same and lands in the same place, so engineers search one system instead of many during an incident.

4.10 Metrics Store

What it is: A time-series database that stores numbers with timestamps and labels, built for fast charts and alert calculations.

> **In everyday terms:** A fitness tracker's history. It can instantly show your heart rate for every minute of the last week.

How it works, step by step

1. Receives metric points such as “errors = 12 at 10:42:00 for service=payments, region=west”.
2. Stores them compressed and indexed by labels and time.
3. Answers queries like “error rate per service over the last 15 minutes” in milliseconds.
4. Keeps high-detail data for a short time (for example, 15 days) and summarized data for longer (for example, 13 months) for trend and SLO reports.

Small details that matter

- Avoid labels with millions of unique values (such as user ID). This is called high cardinality and makes the store slow and expensive. Put those values in logs instead.

Why it helps this project

Alerts and dashboards depend on this store being fast and accurate. It is the source of the detection signal.

4.11 Log Store

What it is: A searchable database for structured logs, indexed by fields.

> **In everyday terms:** A library catalog. Instead of reading every book, you search by author, title or year and go straight to the shelf.

How it works, step by step

1. Receives JSON log records from the collector.
2. Indexes key fields: time, service, level, trace_id, error_code, endpoint.
3. Lets engineers run queries such as: level = ERROR and service = payments in the last 30 minutes, grouped by error_code.
4. Keeps logs for a set retention period, for example 30 days hot and longer in cheaper archive storage.

Small details that matter

- Saved queries for common investigations are linked directly from alerts and runbooks.
- Access is controlled, because logs can contain sensitive operational data.

Why it helps this project

Finding the exact error message and the exact failing requests is often the moment an incident goes from “mystery” to “known cause”.

4.12 Trace Store

What it is: A database that stores traces: the full path of individual requests across services, with timing for each step.

> **In everyday terms:** Package tracking. You can see the parcel left the warehouse at 9:00, reached the hub at 11:00, and got stuck at customs for 3 hours.

How it works, step by step

1. Receives spans from the collector and joins spans that share the same trace ID into one tree.
2. Shows a timeline (waterfall) view: Gateway 1.2 s → Service A 1.15 s → Service B 1.05 s → Database 1.0 s.
3. Lets engineers search for slow or failed traces, and jump from a trace to its logs using the same trace ID.

Small details that matter

- Shows retries as repeated child spans and breaker rejections as tagged spans, so resilience behavior is visible.
- A service map can be built automatically from traces, showing which services call which.

Why it helps this project

In a system with many services, traces answer “where exactly is it slow or failing?” in one view, instead of checking each service one by one.

4.13 Dashboards

What it is: Screens of live charts built from the Metrics Store (and sometimes logs and traces).

> **In everyday terms:** The instrument panel in an airplane cockpit, showing all critical readings at once.

How it works, step by step

1. A top-level **service health dashboard** shows the golden signals for every service, with green, yellow or red status.
2. Each service has its own **detail dashboard**: per-endpoint error rate and latency, dependency health, retry rate, breaker state, CPU and memory.

3. An **SLO dashboard** shows availability against the 99.9% target and how much error budget remains this month.
4. Release markers on charts show when new versions were deployed, making “it broke right after the deploy” obvious.

Small details that matter

- Dashboards follow the same layout for every service, so on-call engineers know where to look even for services they did not build.
- Alerts link directly to the right dashboard, already filtered to the right service and time window.

Why it helps this project

A good dashboard lets an engineer understand the scope of an incident (which service, which region, since when) in the first minute.

4.14 Alerting Engine (Rules, SLOs and Burn Rate)

What it is: A system that continuously checks metrics against rules and fires alerts when something is wrong. The best alerts are tied to **SLOs** (Service Level Objectives), which are reliability targets such as 99.9% of requests succeed.

> **In everyday terms:** A smoke detector. It ignores normal cooking but goes off when there is real smoke, and it is loud enough to wake you.

How it works, step by step

1. Every minute, the engine runs queries on the Metrics Store, for example: percentage of failed requests over the last 5 minutes.
2. It compares results to thresholds. Example rules: error rate above 2% for 5 minutes; p99 latency above 1 s for 10 minutes; circuit breaker open for more than 2 minutes; no data received from a service (it may be completely dead).
3. For SLO-based alerting, it calculates the **error budget burn rate**, meaning how fast the allowed failures are being used up.
4. If a rule stays broken for its required duration (to avoid flapping on single spikes), it fires an alert with severity, summary, affected service, and links to the dashboard, log query and runbook.
5. When the condition clears, it sends a “resolved” notice automatically.

Small details that matter

- **Error budget:** at 99.9%, 0.1% of requests in a 30-day month may fail. That is the budget.

- **Burn rate:** a burn rate of 1 uses the budget exactly over 30 days. A burn rate of 14.4 would use 2% of the monthly budget in 1 hour, which means page someone now. A slow burn (for example, 1 to 3) can open a ticket instead of waking someone.
- **Multi-window checks** (for example, a long and a short window must both be high) reduce false alarms.
- **Severity levels:** Sev 1 or 2 pages a person immediately; Sev 3 or 4 creates a ticket for working hours.
- **Alert hygiene:** every alert must be actionable and have a runbook. Noisy alerts are tuned or deleted, so engineers trust and respond to alerts quickly.

Why it helps this project

Alerts move detection from “a customer complained” to minutes after the problem starts. Burn-rate alerts focus on real user impact, so engineers react to what matters and are not slowed by alarm fatigue.

4.15 Incident Management and Paging

What it is: The system that turns an alert into an incident ticket and reaches the right person on the on-call rotation.

> **In everyday terms:** A hospital's emergency call system: the right doctor's pager goes off, and if they do not answer, it escalates to the next one.

How it works, step by step

1. Receives the alert and creates an incident record with severity, start time and affected service.
2. Groups duplicate alerts into one incident to avoid a flood of pages.
3. Looks up the on-call schedule for the owning team and pages that engineer by phone, SMS or app push.
4. If nobody acknowledges within a set time (for example, 5 to 10 minutes), it escalates to a backup or manager.
5. Records timestamps for every step (alert, acknowledge, mitigate, resolve). These timestamps are how MTTR is measured.

Small details that matter

- For large incidents, an incident commander coordinates and a communications lead updates stakeholders and the status page.

Why it helps this project

Fast, reliable paging shrinks the acknowledge stage, and the recorded timestamps make MTTR measurable so the 40% improvement can be proven.

4.16 On-call Engineer (Triage and Diagnosis)

What it is: The engineer on duty who responds to incidents. With the new tooling, they follow a repeatable investigation path instead of guessing.

> **In everyday terms:** A doctor with a full set of test results already on the desk, instead of having to order and wait for tests.

How it works, step by step

1. Acknowledges the page and opens the alert, which links to the dashboard, log query and runbook.
2. **Scope:** on the dashboard, checks which services, endpoints and regions are affected and when it started. Checks for a recent deployment marker.
3. **Locate:** opens a failing trace to see which hop is slow or failing. Checks circuit-breaker states to see if a dependency is cut off.
4. **Explain:** searches logs by trace ID and error code to read the exact failure reason.
5. **Decide:** chooses the fastest safe mitigation from the runbook.

Small details that matter

- The priority order is: first stop customer pain (mitigate), then find the deep root cause later.
- Updates the incident record as they go, which feeds the postmortem timeline.

Why it helps this project

The diagnose stage was usually the longest part of MTTR. Consistent dashboards, correlation IDs, traces and structured logs make it much shorter.

4.17 Runbooks and Mitigation (Fix)

What it is: Runbooks are written, step-by-step guides for handling known problems. Mitigation is the action that restores service.

> **In everyday terms:** A fire-drill card on the wall: if this happens, do these steps in this order.

How it works, step by step

1. Each alert links to a runbook with: what the alert means, how to confirm it, likely causes, and exact fix steps.

2. Common mitigations: **roll back** the last deployment; **fail over** traffic to a healthy region; **turn off a feature** using a feature flag; **scale out** to more instances; **restart** unhealthy instances; **adjust** a timeout or breaker setting.
3. After acting, the engineer watches the same dashboard to confirm error rate and latency return to normal, and that the breaker returns to CLOSED.
4. The incident is marked mitigated, then resolved once stable.

Small details that matter

- Some runbook steps can be automated (auto-rollback when error rate spikes after a deploy).
- Runbooks are living documents updated after each postmortem.

Why it helps this project

Engineers do not have to invent a fix under pressure, which shortens the mitigate stage and reduces mistakes.

4.18 Postmortem and Continuous Learning

What it is: A blame-free written review after each significant incident, producing concrete improvements.

> **In everyday terms:** An airline's incident review after a near-miss: the goal is to make the system safer, not to blame the pilot.

How it works, step by step

1. Build a timeline from incident timestamps, alerts, logs and chat.
2. Measure each MTTR stage (detect, acknowledge, diagnose, mitigate, recover) to see where time was lost.
3. Find the root cause and contributing factors, often using a “5 whys” approach.
4. Create action items with owners and due dates: add a missing alert, add a log field, tune a breaker threshold, update a runbook, fix the bug.
5. Track action items to completion and review MTTR trends monthly.

Small details that matter

- Questions to ask every time: Did we detect it before customers? Did the alert link to the right place? Was any data missing? Could a retry or breaker have prevented it?

Why it helps this project

This feedback loop (arrow 19 in the diagram) is why MTTR kept improving over time rather than improving once. Each incident makes the next one shorter.

5. Workflows, End to End

Each workflow below walks through one real situation from start to finish. Diagram arrow numbers are shown in brackets so you can follow along on the picture.

5.1 Workflow 1: A normal, healthy request

A customer places an order and everything works. This is the baseline that all other workflows build on.

Step	What happens	Simple explanation
1	User taps “Place order”. The app sends an HTTPS request. [1]	The customer asks the system to do something.
2	API Gateway authenticates the request, creates trace ID 4f2a9c, picks a healthy Service A instance, and forwards the request. [2]	The front door labels the visitor and sends them to an open desk.
3	Service A starts a span, logs INFO “order_received” with trace_id, validates the cart.	Orders department starts a stopwatch and notes the visit in its diary.
4	Service A calls Service B with an 800 ms timeout, through its circuit breaker (CLOSED), passing trace ID. [3]	Orders asks Payments for help, with rules about how long to wait.
5	Service B starts a child span and calls the database and bank API, each with its own timeout and breaker. [4]	Payments talks to its suppliers.
6	All calls succeed. Spans close with durations; counters increase; INFO logs record success.	Every department writes down that the job went well and how long it took.
7	The response flows back: B → A → Gateway → User, with the trace ID in the response header.	The customer gets their confirmation.
8	In the background, A and B send metrics, logs and spans to the Collector. [5][6]	Reports go to the central mailroom without slowing the customer.
9	Collector enriches, filters and routes data to the three stores. [7][8][9]	The mailroom sorts and files everything.
10	Dashboards update; the alerting engine evaluates rules and finds nothing wrong. [10][11]	The health screens stay green; the smoke detector stays quiet.

Result: Customer is served quickly. A complete record of the request exists, ready if anything ever needs to be investigated.

5.2 Workflow 2: The life of a telemetry signal

This follows one piece of data (an error count, a log line and a span) from the moment it is created to the moment a human sees it.

Step	What happens	Simple explanation
1	Inside the service, the SDK increments http_errors_total, the logger writes a JSON line, and a span ends with status=ERROR.	The service writes down what happened, in three formats.
2	Data is buffered in memory and sent in a batch every few seconds. [5][6]	Notes are collected into an envelope and posted together.
3	Collector receives, adds region and cluster labels, masks sensitive fields, keeps all error data, samples routine success traces.	The mailroom stamps, cleans and sorts.
4	Metrics go to the Metrics Store, logs to the Log Store, traces to the Trace Store. [7][8][9]	Each type is filed in its own cabinet.
5	Within seconds, dashboards show the new data point and the alert engine includes it in its next check. [10][11]	The data is now visible and being watched.
6	An engineer can later query any of the three stores and jump between them using the trace ID. [14][15]	All three records can be linked back to the same customer request.

Result: Signals travel from code to screen in seconds, are consistent across services, and are not lost even if a store briefly fails.

5.3 Workflow 3: A short network blip handled by retries

The bank API drops one connection for a fraction of a second. Without retries, the customer would see an error.

Step	What happens	Simple explanation
1	Service B calls the bank API. The connection resets after 40 ms. [4]	The phone line crackles and drops.
2	Retry policy checks: is this transient? Yes (connection reset). Is the call idempotent? Yes, an idempotency key is attached.	It is safe to call again without charging twice.
3	Logger writes WARN “retry_attempt” with attempt=2, trace_id; retry counter increases.	A note is made that a second try was needed.
4	Wait 100 ms plus random jitter, then retry.	Pause briefly, so not everyone redials at the same moment.
5	Second attempt succeeds in 120 ms.	The call goes through.
6	Span shows two child attempts; the overall request succeeds well within its deadline.	The record shows a hiccup that was handled.
7	Breaker records one failure and one success; failure rate stays far below the trip threshold, so it stays CLOSED.	The safety switch sees nothing serious.

> **Result:** The customer never notices. No alert fires. The retry metric shows the blip, so if blips become frequent, the team will see a trend before it becomes an outage.

5.4 Workflow 4: A dependency outage contained by the circuit breaker

The bank API goes fully down for 15 minutes. This shows how retries and the breaker cooperate to protect the whole site.

Step	What happens	Simple explanation
1	Calls to the bank API start timing out after 800 ms each. [4]	The supplier stops answering.
2	Retries try up to 3 times per request, but all fail. Retry and error metrics rise sharply.	Redialing does not help this time.
3	Within the rolling window, the failure rate passes 50% over 20+ calls. Breaker for bank-api switches CLOSED → OPEN.	The safety switch trips.

Step	What happens	Simple explanation
4	Logger writes ERROR “circuit_opened” dependency=bank-api; breaker-state metric becomes OPEN.	A clear note says exactly which supplier is cut off.
5	New payment requests skip the bank call and use the fallback: accept the order as “payment pending” and queue it for later processing.	The shop takes orders and promises to process payment shortly, instead of turning customers away.
6	Service B's threads and connections are freed instantly instead of waiting on timeouts, so other features (order history, product pages) stay fast.	One broken supplier does not block the whole shop.
7	Alert “bank-api breaker open > 2 min” and the burn-rate alert fire, and the on-call engineer is paged. See Workflow 5. [11][12][13]	Someone is told right away.
8	Every 30 seconds the breaker goes HALF-OPEN and sends a few test calls. They fail, so it returns to OPEN.	It checks carefully whether the supplier is back.
9	When the bank API recovers, test calls succeed and the breaker returns to CLOSED. Queued payments are processed.	Normal service resumes on its own.

> **Result:** Instead of a full site outage, one feature was degraded gracefully. Fewer failed requests means less error budget used, protecting 99.9% availability, and the breaker metric pointed straight at the cause.

5.5 Workflow 5: Incident detection to recovery (the MTTR workflow)

A new release of Service A introduces a bug that makes 8% of checkout requests fail. This is the workflow that the 40% MTTR improvement is measured on.

Step	What happens	Simple explanation
1	**10:42** Version 2.3 of Service A is deployed. Error rate on /checkout jumps from 0.1% to 8%.	Problem starts. The MTTR clock starts.
2	Metrics reach the Metrics Store within seconds. Dashboard turns red with a deploy marker at 10:42. [5][7][10]	The numbers show it immediately.

Step	What happens	Simple explanation
3	**10:44** Burn-rate alert: error budget burning at about 80x normal (short and long windows both high). Sev 2 alert fires. [11]	Detect stage complete in about 2 minutes.
4	Incident created; on-call engineer paged with links to dashboard, log query and runbook. [12][13]	The right person is reached with everything they need.
5	**10:46** Engineer acknowledges.	Acknowledge stage complete.
6	Dashboard shows: only Service A, only /checkout, all regions, starting exactly at the 10:42 deploy.	Scope found in one look: a code change, not a dependency.
7	Engineer opens a failed trace: failure is inside Service A itself, not B or the database (all breakers CLOSED). [15]	Location confirmed.
8	Log search: service=orders level=ERROR error_code=* since 10:42 shows NULL_DISCOUNT_CODE from the new version. [14]	Exact cause visible.
9	**10:53** Runbook step: “Errors start right after a deploy → roll back.” Engineer rolls back to version 2.2. [16][17]	Mitigation chosen without guesswork.
10	**10:57** Dashboard shows error rate back to 0.1%; alert resolves automatically.	Recovery verified. MTTR clock stops: about 15 minutes.
11	Incident timestamps are saved for MTTR reporting.	The improvement is measurable, not a feeling.

> **Result:** Without the project, this incident might have been found by customer complaints and diagnosed by reading raw logs on servers. With it, every stage was shorter, which is exactly how a 40% reduction in average MTTR is achieved.

5.6 Workflow 6: Learning loop after the incident

After service is restored, the team makes sure the next similar incident is shorter or never happens.

Step	What happens	Simple explanation
1	Engineer opens a postmortem and builds the timeline from incident timestamps, alerts and logs. [18]	Write down exactly what happened and when.
2	Team measures each MTTR stage: detect 2 min, acknowledge 2 min, diagnose 7 min, mitigate and verify 4 min.	See which stage took longest.
3	Root cause: missing null check on discount codes; tests did not cover empty discounts.	Find why it really happened.
4	Action items: add unit test; add canary deployment with automatic rollback on error spike; add a log field for discount type; update runbook.	Turn lessons into specific tasks with owners.
5	New or tuned alerts and runbook steps are added to the alerting engine and documentation. [19]	Improve the smoke detector and the fire-drill card.
6	MTTR and availability trends are reviewed monthly against the 99.9% SLO.	Keep checking that things are getting better.

Result: Each incident improves detection, diagnosis or prevention. Over many incidents, this loop is what sustains the lower MTTR.

6. How the Results Are Measured

6.1 Measuring MTTR

1. For each incident, record the start time (when impact began, read from metrics) and the recovery time (when metrics returned to normal).
2. MTTR = total recovery time across all incidents ÷ number of incidents, over the same type of period (for example, a quarter).
3. Compare the period before the project with the period after it. A 40% reduction means the new average is 60% of the old one, for example 50 minutes down to 30 minutes.
4. Also track each stage separately (time to detect, time to acknowledge, time to mitigate) to know which change helped most.

6.2 Measuring 99.9% availability

1. Define a good request: it returned a success response and was faster than an agreed limit, for example under 1 second.
2. Availability = good requests ÷ total requests, measured over 30 days from gateway metrics.
3. 99.9% means at most 1 bad request in every 1,000. In time terms, about 43.8 minutes of full downtime per month.
4. Retries and circuit breakers turn many would-be bad requests into good or gracefully degraded ones, and faster recovery shortens the time spent below target.

Availability target	Allowed downtime per month	Allowed downtime per year
99%	about 7.3 hours	about 3.65 days
99.9% (this project)	about 43.8 minutes	about 8.8 hours
99.99%	about 4.4 minutes	about 52.6 minutes

> **Why lower MTTR protects availability:** Downtime ≈ number of incidents × average recovery time. Cutting recovery time by 40% cuts downtime by about 40% for the same number of incidents, leaving much more room under the 43.8-minute monthly budget.

7. Typical Tools for Each Component

The project statement does not name specific products. The table lists common choices so the design can be mapped to a real stack. “Live Site” is a term used often in Microsoft and Azure teams, so Azure options are listed first.

Component	Azure / Microsoft options	Open-source / other options
Telemetry SDK	Application Insights SDK, Azure Monitor OpenTelemetry Distro	OpenTelemetry SDK, Micrometer, Prometheus client
Structured logging	ILogger with JSON formatting, Serilog	Serilog, Logback/Log4j2 JSON, Python structlog, Winston
Retry and timeout	Polly (.NET), Microsoft.Extensions.Http.Resilience	Resilience4j (Java), tenacity (Python), Envoy or Istio retry policies
Circuit breaker	Polly circuit breaker	Resilience4j, Envoy outlier detection, Istio
Telemetry Collector	Azure Monitor Agent, Data Collection Rules	OpenTelemetry Collector, Fluent Bit, Vector
Metrics Store	Azure Monitor Metrics, Managed Prometheus	Prometheus, Thanos, Mimir, Datadog
Log Store	Log Analytics (KQL queries), Azure Data Explorer	Elasticsearch/OpenSearch, Loki, Splunk
Trace Store	Application Insights (transaction search, application map)	Jaeger, Tempo, Zipkin, Honeycomb
Dashboards	Azure Workbooks, Azure Managed Grafana	Grafana, Kibana
Alerting	Azure Monitor alerts, action groups	Prometheus Alertmanager, Grafana Alerting
Incident and paging	IcM (internal), Azure Monitor action groups to Teams	PagerDuty, Opsgenie, incident.io

8. Explaining It in 30 Seconds

> **Short version:** “Our services were hard to debug and fragile when dependencies misbehaved. I added consistent telemetry and structured JSON logging with correlation IDs across services, built SLO-based burn-rate alerts with linked dashboards and runbooks, and wrapped dependency calls in timeouts, retries with backoff and jitter, and circuit breakers with fallbacks. Incidents were detected in minutes, diagnosed by following one request ID end to end, and contained before they spread. Average time to recover dropped 40% while we held 99.9% availability.”

Likely follow-up questions

- **How did you avoid retry storms?** Retries only on transient errors and idempotent calls, capped attempts, exponential backoff with jitter, a retry budget, and retries at one layer only.
- **How did you choose breaker thresholds?** Started from normal error rates and traffic seen in metrics, required a minimum number of calls before tripping, and tuned after load tests and postmortems.
- **How did you reduce alert noise?** Alerted on user-facing SLO burn rate rather than every CPU spike, used multi-window checks, and required every paging alert to have a runbook.
- **How did you prove the 40%?** Incident timestamps from the incident system, comparing average recovery time before and after rollout over equal periods.